

Communications and Information Engineering Program

DSP (CIE 247), Spring 2025

Spectral Estimation and Digital Filtering of an Audio Signal

Omar Tamer Samir

202300713

Part 1: Spectral Estimation

-Introduction:

In this part of the project, we analyze the frequency content of an audio signal through spectral estimation techniques. A sinusoidal interference is added to the original signal, and its effect is examined using the Welch method. The goal is to understand how different parameters—such as FFT size, window type, and overlap—impact the resolution and accuracy of the power spectrum.

1. Reading the audio file

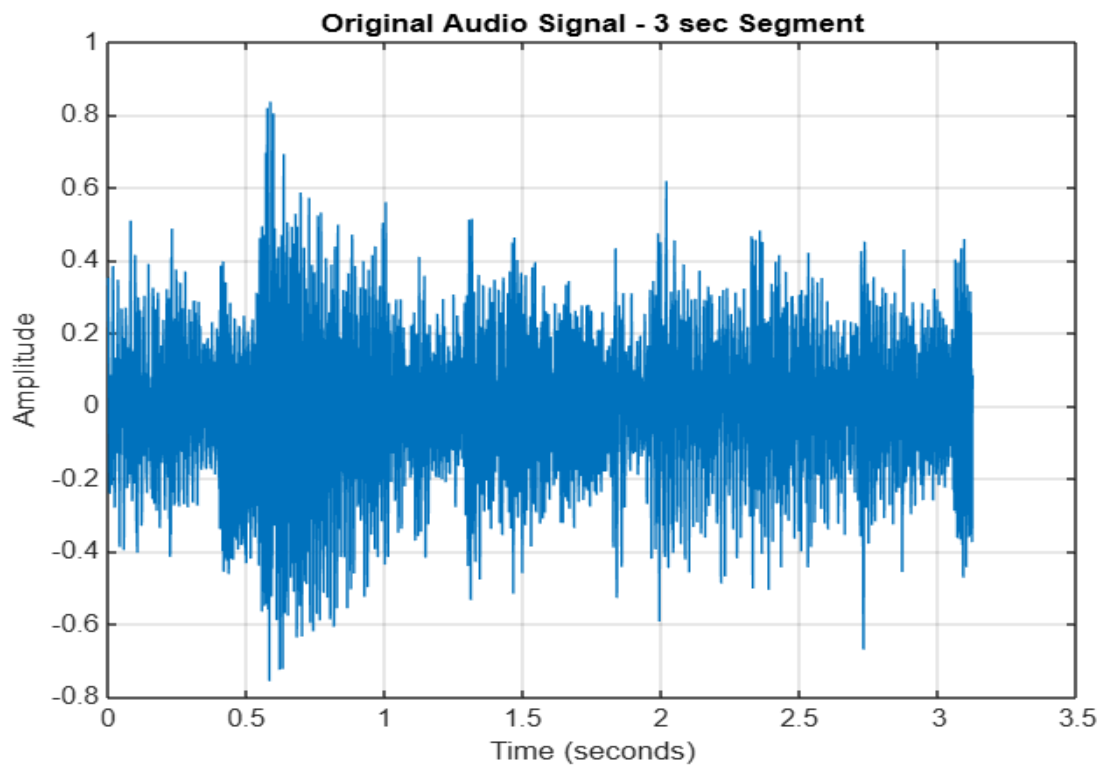
```
[X, Fs] = audioread('music_test_fayrouz.mp3'); % Fs = 32000 Hz
```

2. Capturing 3-seconds from audio

```
Y = X(3e5:4e5, 1); %select ~3 sec segment(at Fs=32000) from 100,000 samples
```

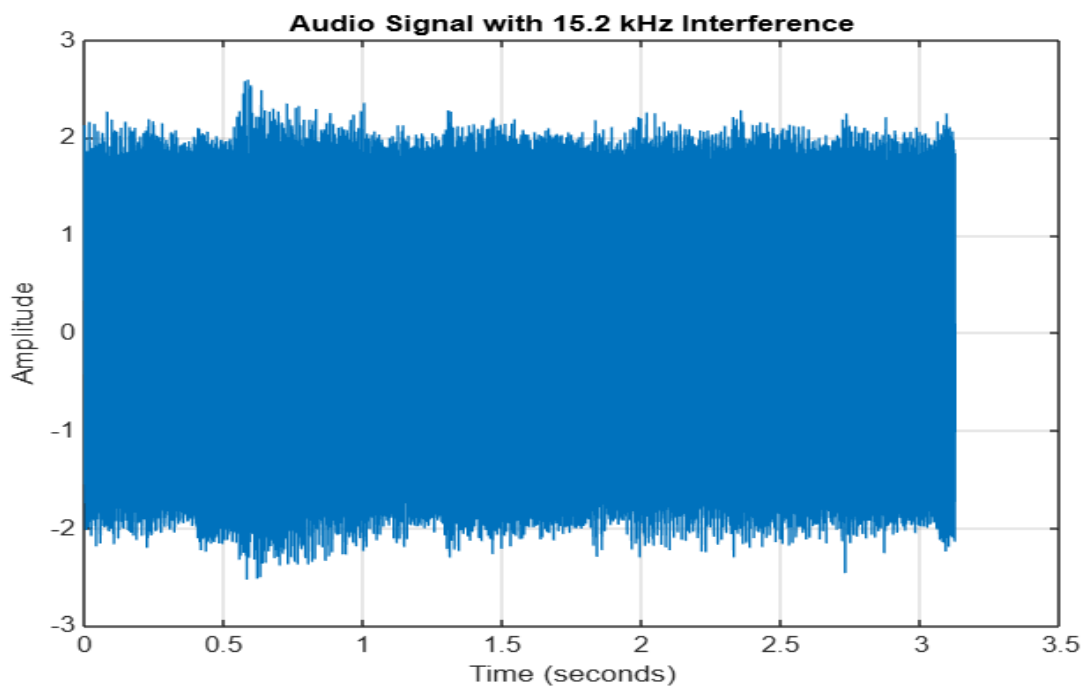
3. Plot the audio signal versus time in seconds

```
% Plot the original audio segment
n = 0:length(Y)-1;
ts = 1/Fs;
t = n * ts;
figure;
plot(t, Y);
xlabel('Time (seconds)');
ylabel('Amplitude');
title('Original Audio Signal - 3 sec Segment');
grid on;
```



4. Generate a sinusoidal interference tone at 15.2 KHz

```
tone_freq = 15.2e3;  
A = 1.8;  
Y_Interference = A * sin(2 * pi * tone_freq * t);  
Y_total = Y + Y_Interference'; % Add interference tone to the audio segment  
  
%Plot the audio with interference  
figure;  
plot(t, Y_total);  
xlabel('Time (seconds)');  
ylabel('Amplitude');  
title('Audio Signal with 15.2 kHz Interference');  
grid on;
```



5. Plotting the interference signal using the Welch spectral estimation

```
function Plot_Spectrum(signal, Fs, FFTSize, WindowSize, win_type, overlap)

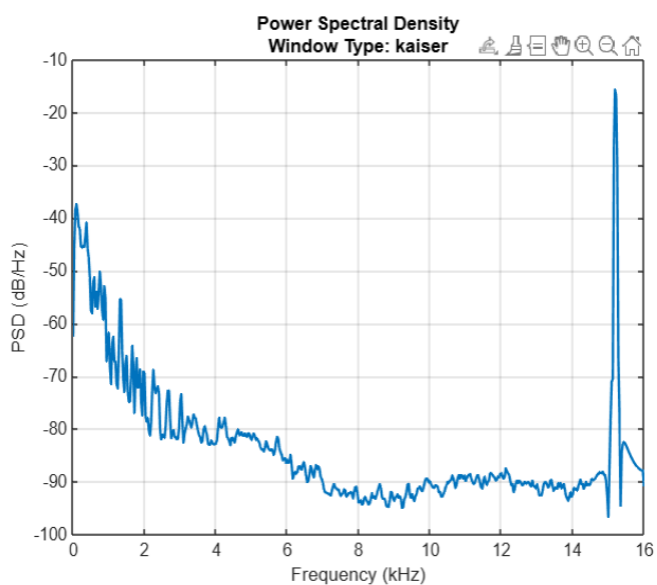
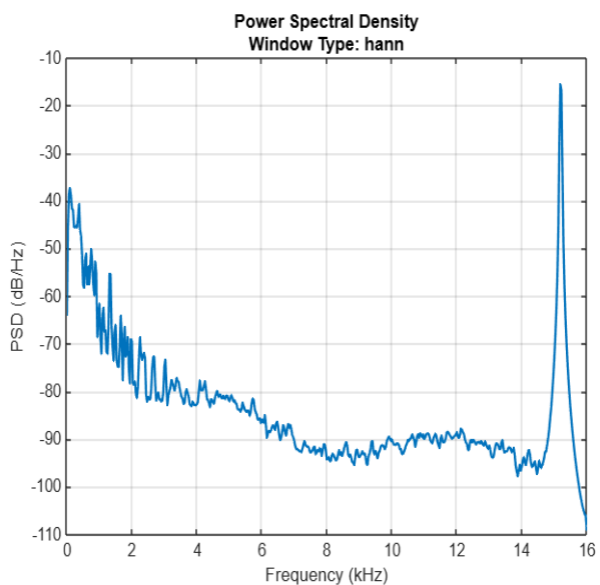
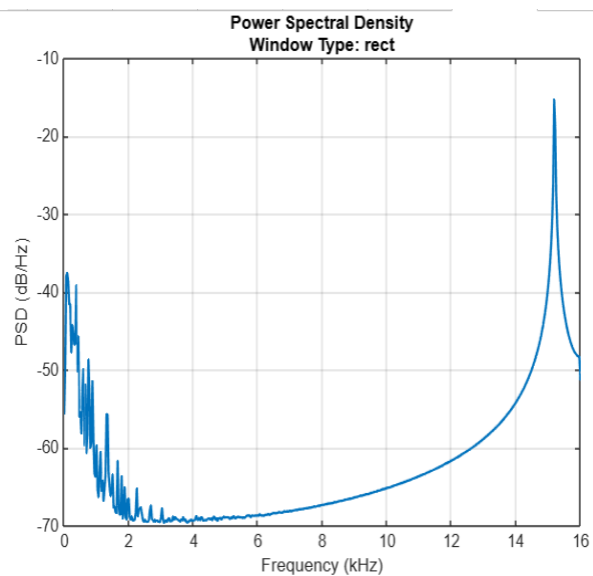
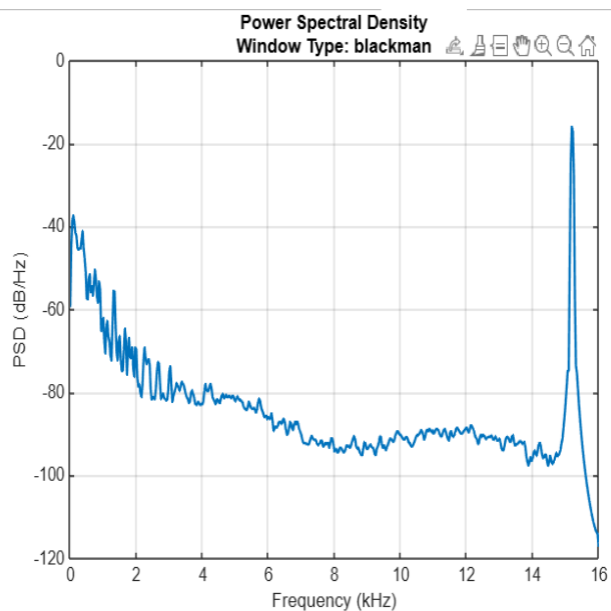
    switch win_type
        case 'kaiser'
            beta = 7;
            window = kaiser(WindowSize, beta);
        case 'hann'
            window = hann(WindowSize);
        case 'blackman'
            window = blackman(WindowSize);
        case 'rect'
            window = rectwin(WindowSize);
        otherwise
            error('Unknown window type');
    end

    % Calculate PSD using Welch's method
    PSD = pwelch(signal, window, overlap * FFTSize, FFTSize, Fs);
    Freq = 0:Fs/FFTSize:Fs/2;
    Freq = Freq / 1e3;

    % Plot
    figure;
    plot(Freq, 10 * log10(PSD), 'LineWidth', 1.5);
    xlabel('Frequency (kHz)');
    ylabel('PSD (dB/Hz)');
    title(sprintf('Power Spectral Density\nWindow Type: %s', win_type));
    grid on;
end
```

```
% Spectral Analysis with different Welch parameters
FFTSize = 1024;
WindowSize = 1024;
window_types = {'kaiser', 'hann', 'blackman', 'rect'};
overlap_percentage = 0.5;

for i = 1:length(window_types)
    Plot_Spectrum(Y_total, Fs, FFTSize, WindowSize, window_types{i}, overlap_percentage)
end
```



6. Effect of Welch Method Parameters on the Power Spectrum

1. FFT Size

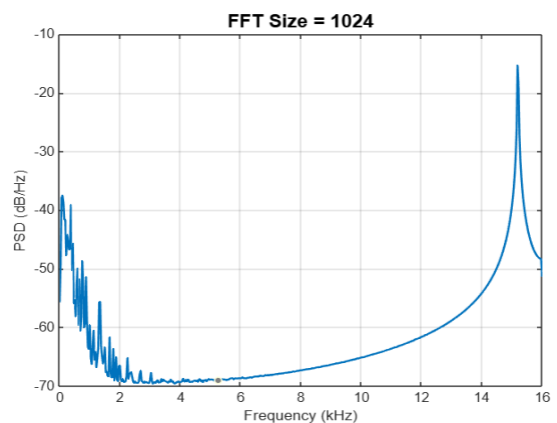
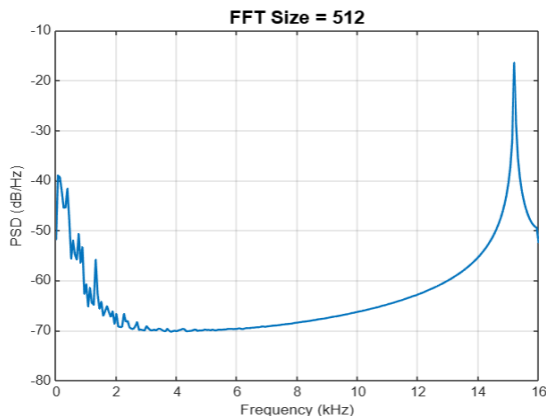
Effect: Determines the frequency resolution.

- A **larger FFT size** increases frequency resolution (narrower frequency bins), which reveals sharp and narrow peaks more clearly.
- A **smaller FFT size** reduces resolution, resulting in a blurrier spectrum but lower computational cost.

Illustration:

-Generate PSD plots for `fft_size = (512), (1024), and (4096)` for rect window.

-Notice how the interference tone at 15.2 kHz becomes increasingly sharp and well-defined as FFT size increases.



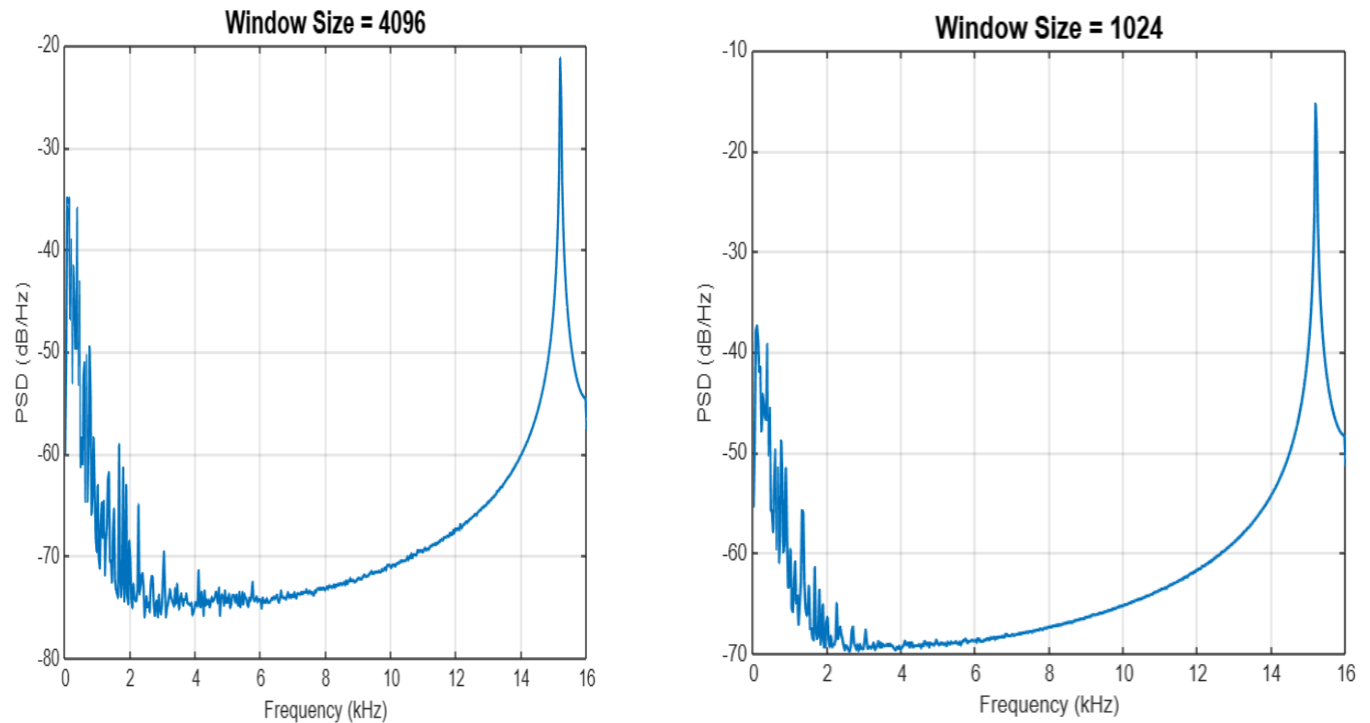
2. Window Size

Effect: Balances between resolution and averaging.

- A **longer window** provides better frequency resolution but fewer segments for averaging, which may introduce noise.
- A **shorter window** allows more averaging (smoother PSD), but sacrifices resolution.

Illustration:

Plot PSDs using window_size = (1024) and (4096) for rect window while keeping FFT size constant.



This code clearly shows how:

- **Small windows** smooth the spectrum (less resolution, more averaging),
- **Large windows** reveal sharper frequency components (higher resolution, less averaging).

3. Window Type

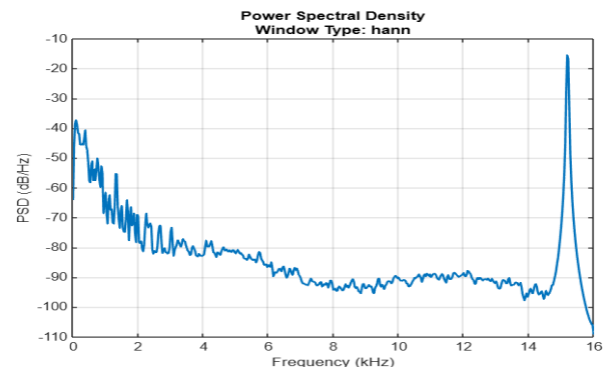
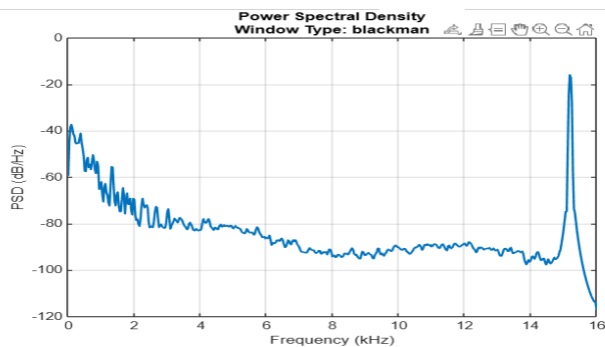
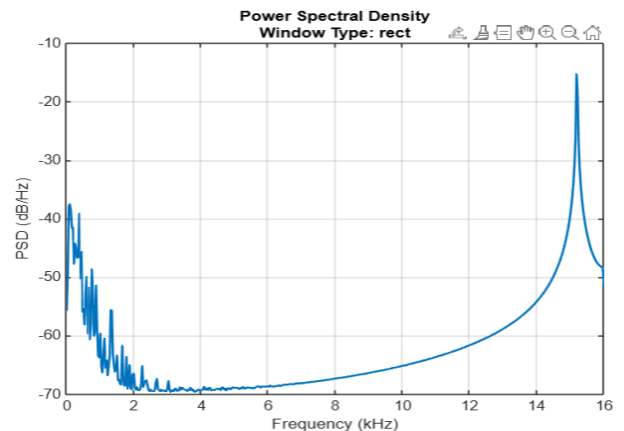
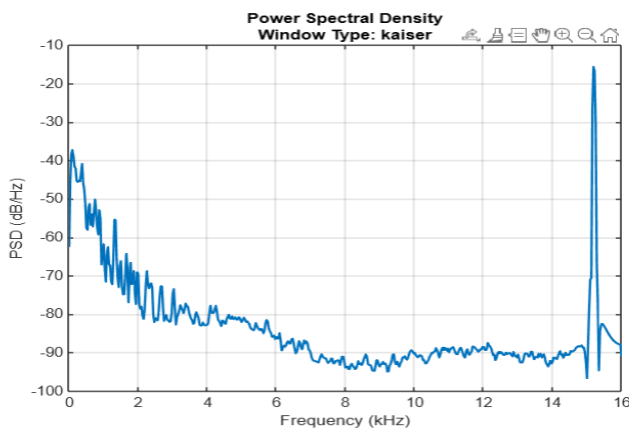
Effect: Controls **spectral leakage**.

- **Rectangular:** High resolution but suffers from significant spectral leakage.
- **Hanning:** Balance between leakage reduction and resolution.
- **Blackman-Harris:** Excellent leakage suppression, but wide main lobe reduces resolution.

- **Kaiser Window:** The Kaiser window offers a flexible trade-off between spectral leakage and frequency resolution. You can control the window's characteristics by adjusting a parameter (β).
 - For small β (low values), it behaves like a rectangular window with good frequency resolution but suffers from leakage.
 - For large β (high values), it behaves like a Blackman-Harris window with lower resolution but very little leakage.

Illustration:

Use identical settings with different windows. Compare how side lobes are suppressed in Kaiser, Hanning, and Blackman-Harris windows versus the rectangular one.



- **Rectangular window:** shows higher side lobes (more spectral leakage).
- **Hanning** :reduce the leakage while preserving resolution fairly well.
- **Blackman-Harris** : minimizes leakage significantly but results in broader peaks.

- **Kaiser window ($\beta = 7$)** provides a good balance, significantly reducing spectral leakage while maintaining a reasonable resolution, offering better performance than rectangular and Hamming windows, but not as sharp as Blackman-Harris.

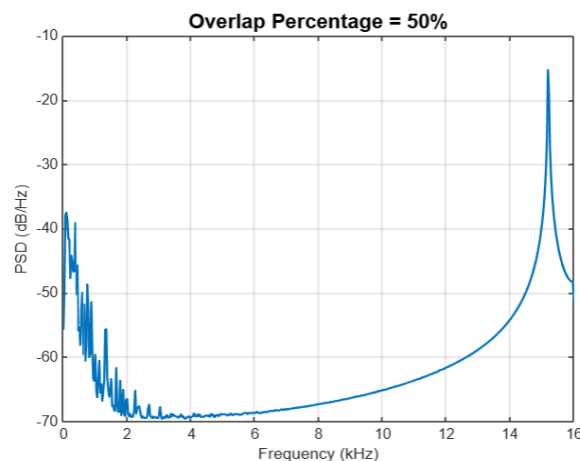
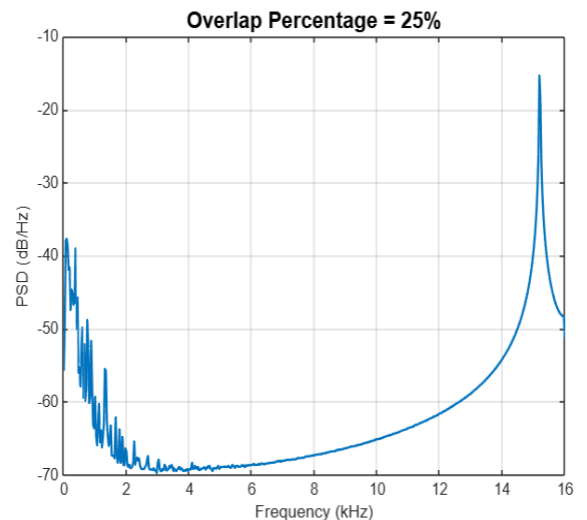
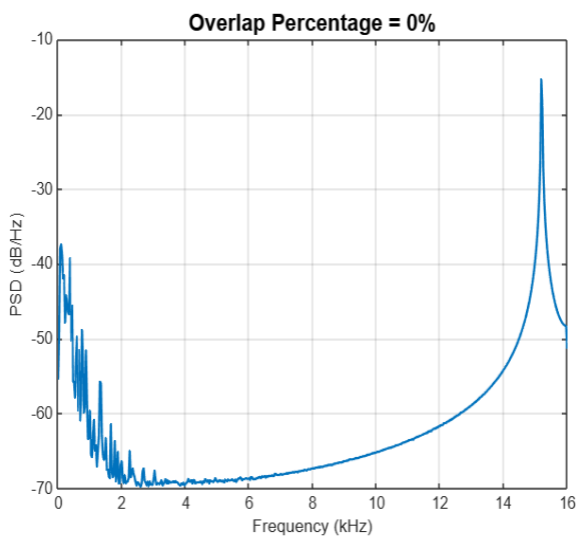
4. Overlap Percentage

Effect: Impacts the **smoothness and statistical stability** of the estimate.

- Higher overlap provides more averaging, reducing variance and making the spectrum smoother.
- However, excessive overlap increases computational load with minimal improvement.

Illustration:

Plot PSDs with 0%, 50%, and 75% overlap. As overlap increases, the power spectrum becomes more stable and less noisy.



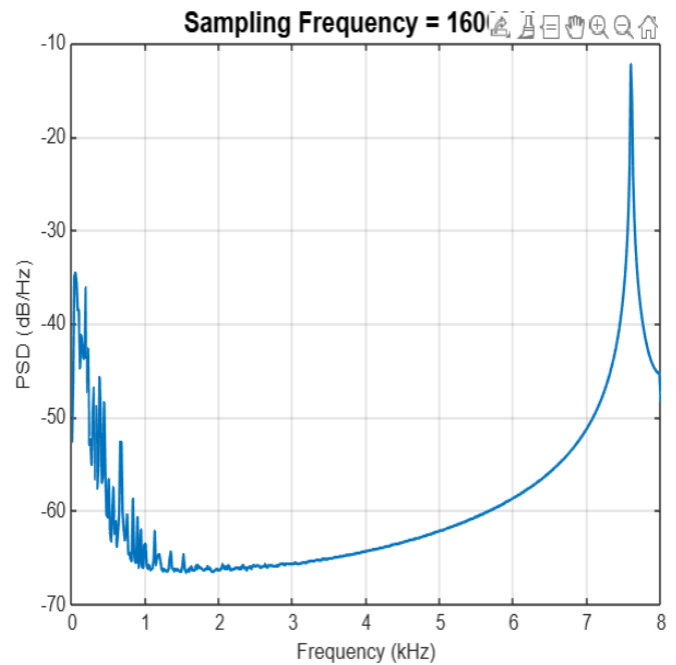
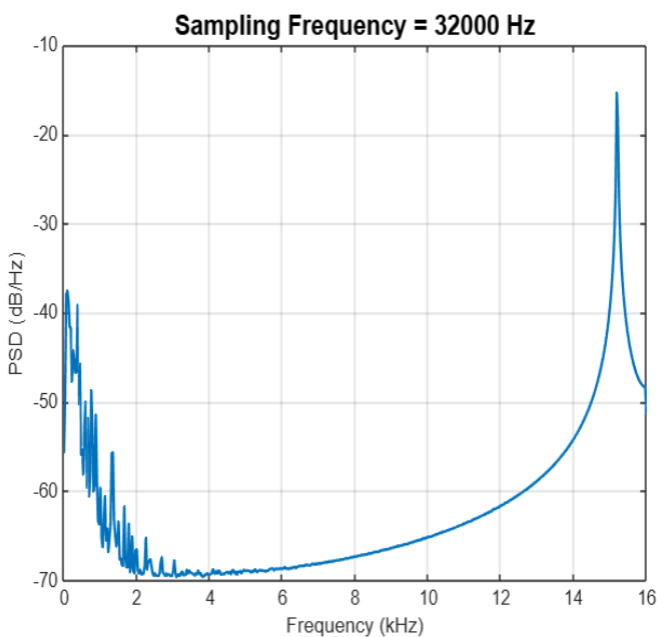
5. Sampling Frequency (F_s)

Effect: Defines the **frequency axis scale**.

- Does **not change the shape** of the PSD but determines the x-axis in Hz.
- Downsampling compresses the spectrum; high F_s spreads it out.

Illustration:

- Plot the welch spectrum for $F_s=16\text{KHz}$ and $F_s=32\text{KHz}$ to compare their effects.



- **Increasing F_s doesn't affect the actual frequency content of the signal, but it makes the frequency graph clearer and more detailed.**

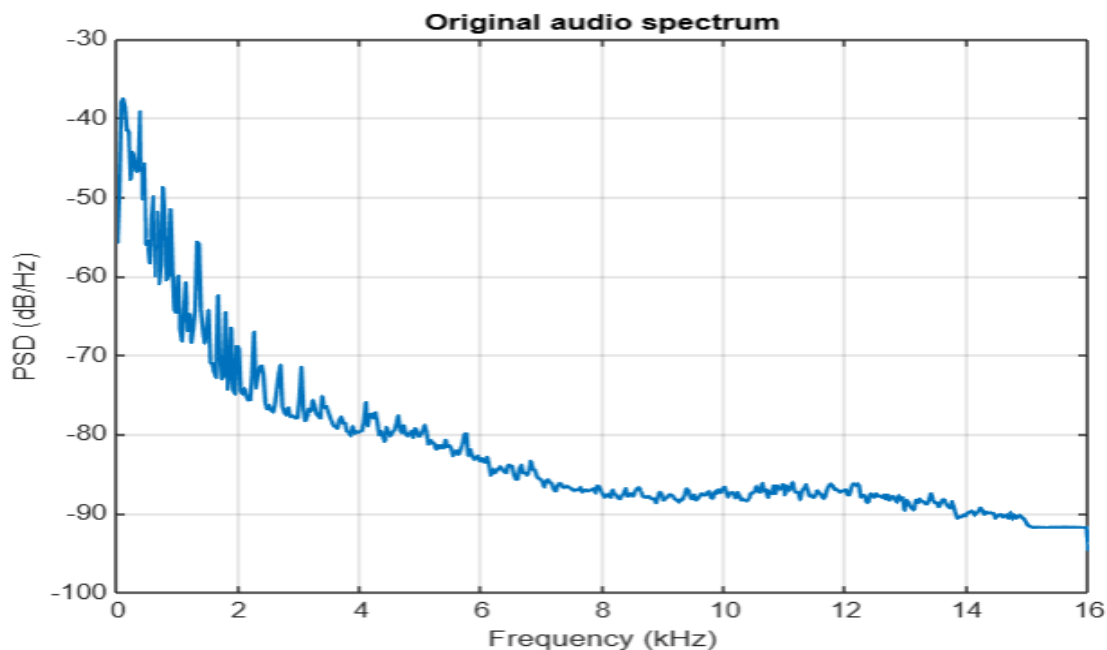
- With a higher F_s , the graph looks more "compressed" because it shows finer details in the frequency range.
- A lower F_s might cause you to lose high-frequency information and could lead to aliasing if the signal contains frequencies higher than half of F_s .

PART2-Digital filter design

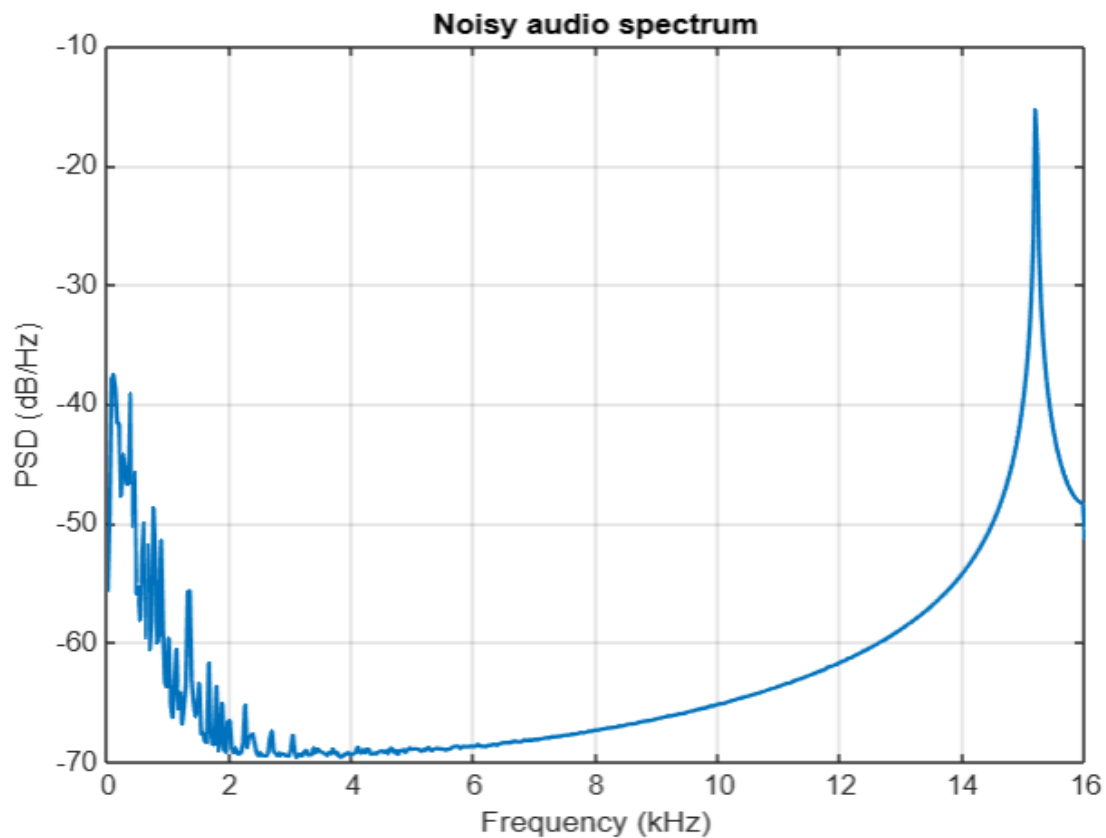
1)Problem Statement

We have an original audio signal that has been affected by unwanted noise at a frequency of **15.2 kHz**. The goal is to **remove this interference** and **recover the clean original audio**. The **spectrum representation** of the noisy signal clearly shows a sharp peak at 15.2 kHz, confirming the presence of the interference. To solve this problem, we designed a **Low pass filter with cut off frequency =15.2KHz** that removes the noise while preserving the rest of the audio content.

->Original audio spectrum



->Noisy audio spectrum



2)Constraints in the Digital Filtering Problem:

1-Type of filter:

FIR filter [Low pass filter] to ensure a linear phase response, which helps preserve the shape and quality of the audio signal during filtering.

2. Maximum Filter Order Constraint:

The filter order (N) is set to not exceed **250** samples. This value provides a good balance between frequency resolution and computational efficiency. It is high enough to effectively remove the

narrowband interference at 15.2 kHz, while keeping memory usage and processing requirements within practical limits.

3- Latency Constraint:

FIR filters introduce a latency of approximately **$N/2$** samples. For **$N=150$ samples**, this results in a delay of about **75 samples**. This level of latency is acceptable for the application, as it does not significantly impact performance or audio quality.

3)Filter specifications:

- Passband Frequency: 14.6 kHz

- This is the lower edge of the passband, where the filter allows original audio frequencies to pass through with minimal distortion.

- Stopband Frequency: 15 kHz

- This is just below the target frequency (15.2 kHz), where the filter begins attenuating unwanted interference.

- Transition Region: 14.6 kHz to 15 kHz (600 Hz)

- The transition region allows for a smooth shift from the passband to the stopband, providing efficient interference removal at 15.2 kHz.

- Passband Ripple: Maximum of 1 dB

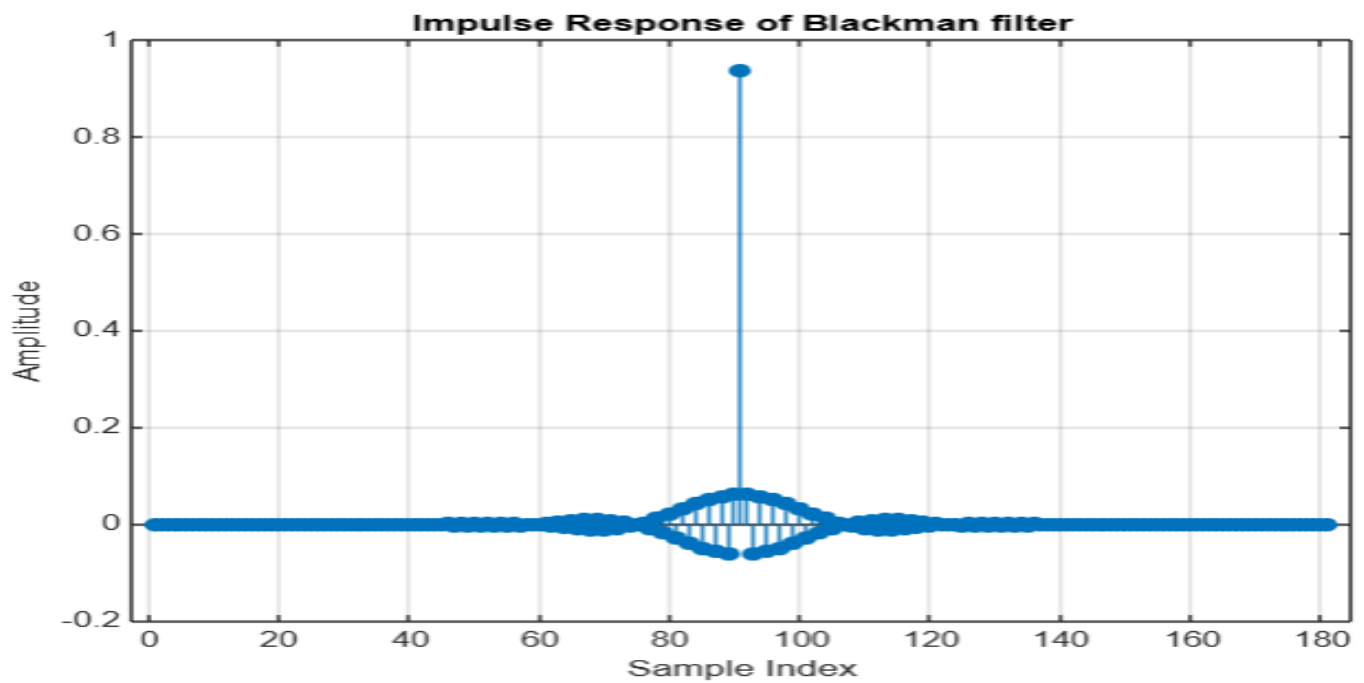
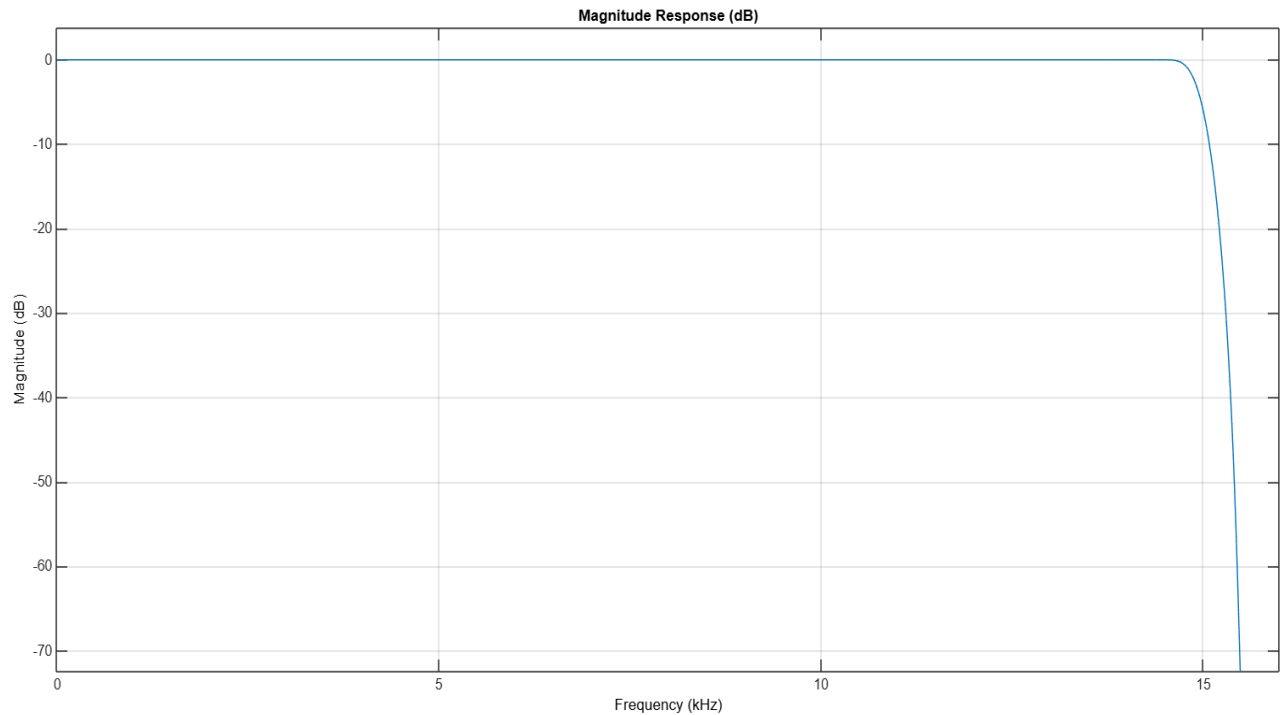
- Ensures minimal distortion in the passband.

- Stopband Attenuation: Minimum 60 dB

- Effectively reduces interference at 15.2 kHz.

4) Filter Design Methods

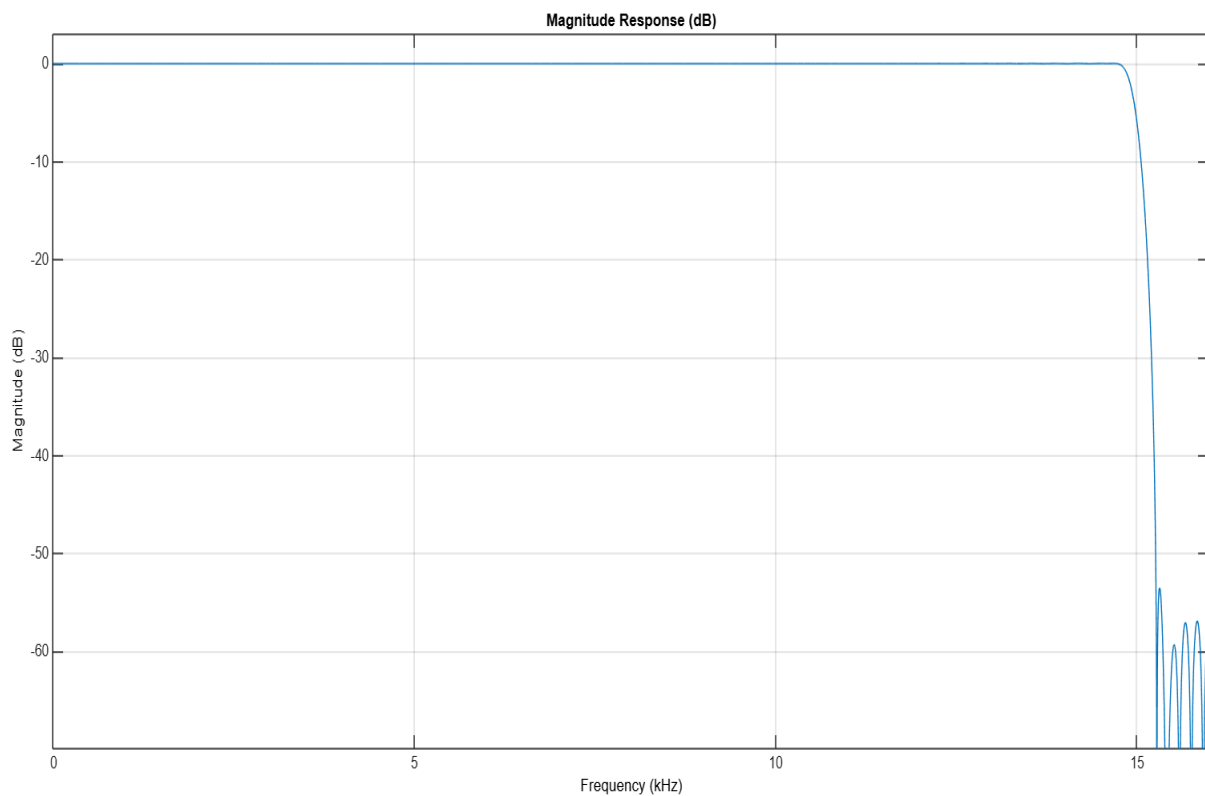
1. Blackman Window filter(N=180):

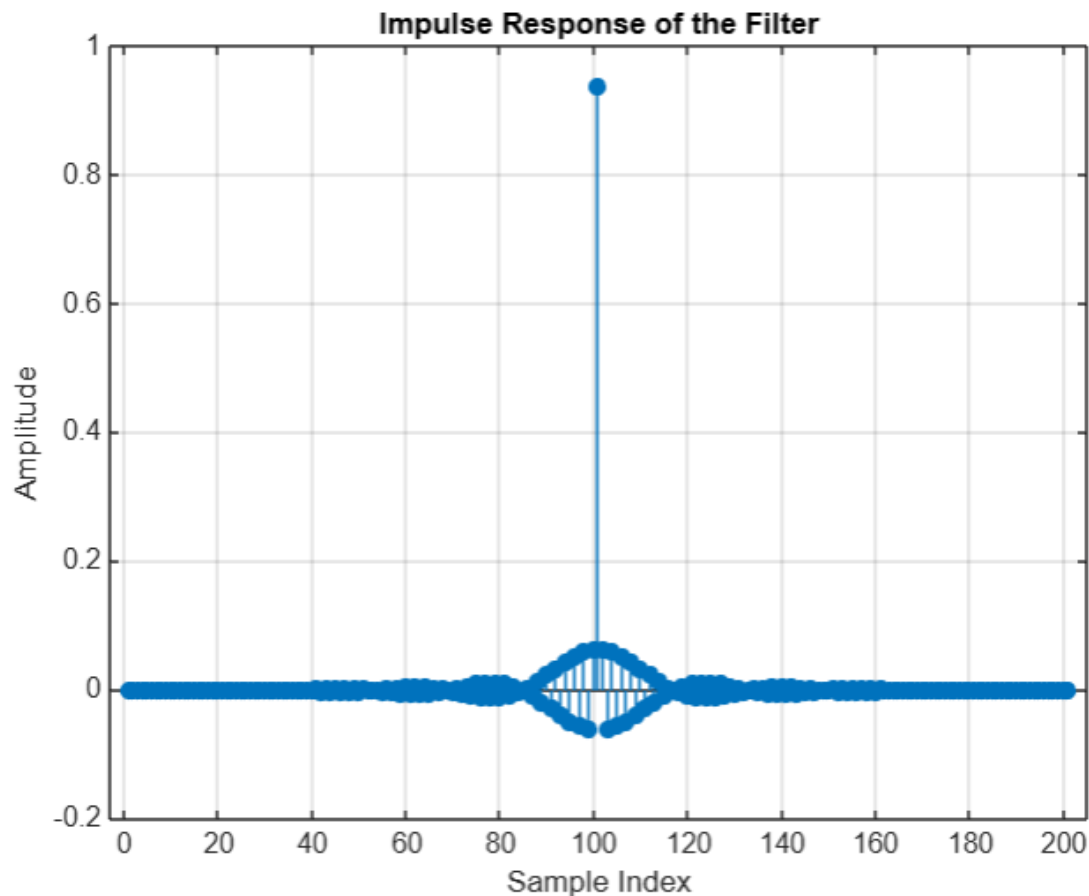


Observations:

- **Passband:** Very flat and clean with minimal ripple (~ 0 dB)
- **Transition Band:** Wider compared to the other filters (500–600 Hz)
- **Stopband:** Good attenuation, better than Hamming. (70–75 dB)
- **Overall:** Meets all constraints.

2. Hamming Window Filter (N=200):

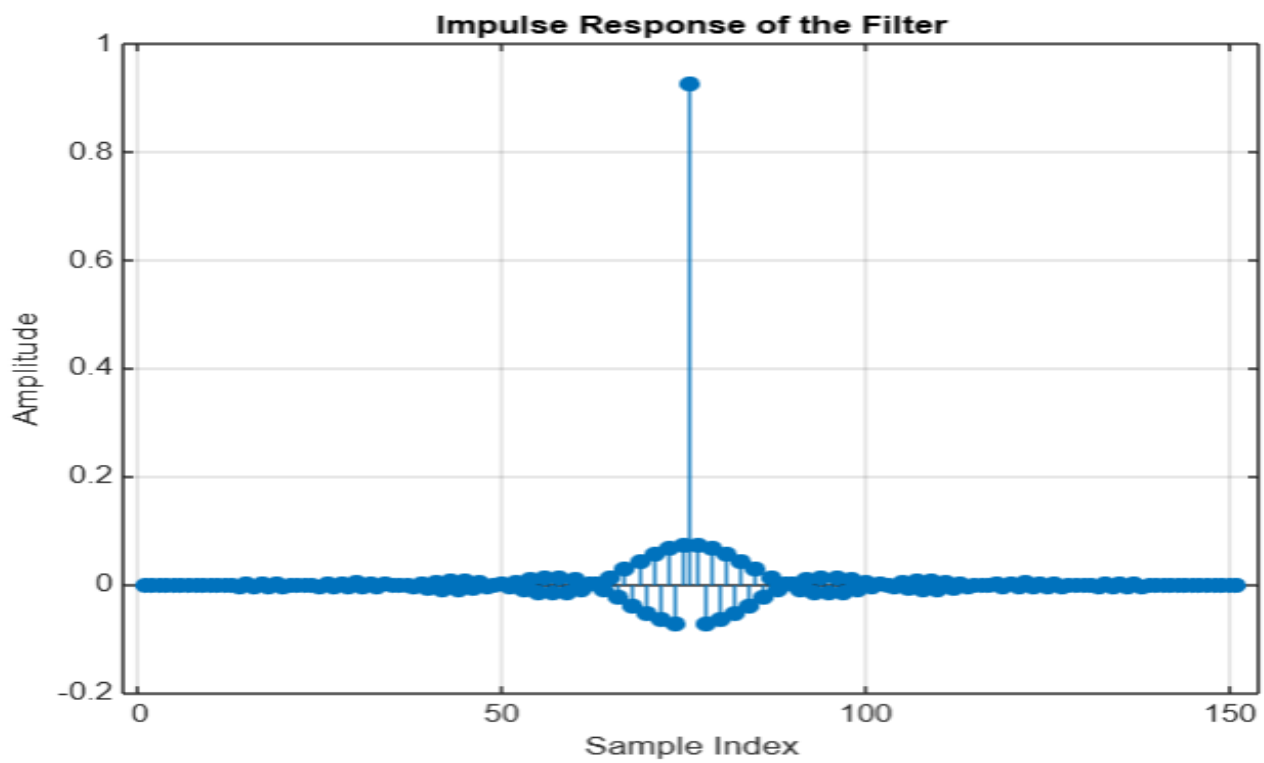
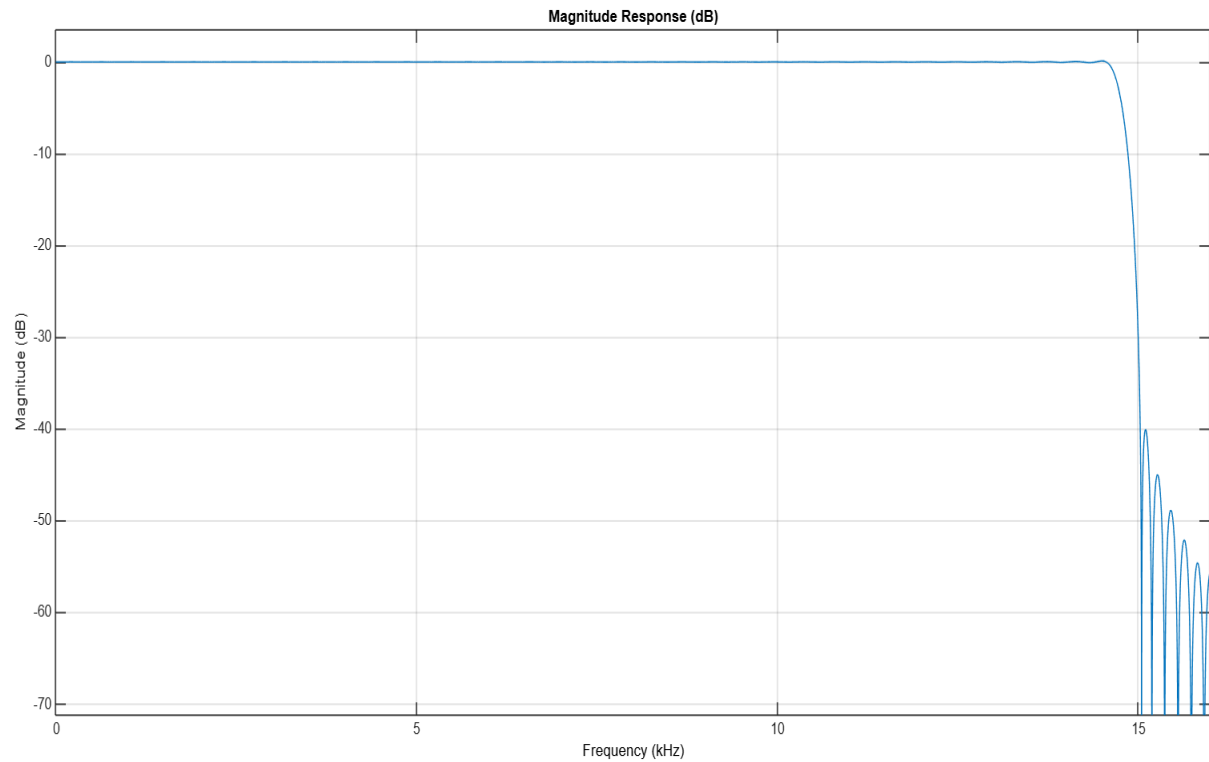




-Observations:

- **Passband:** Very flat and clean, similar to Blackman(~0dB)
- **Transition Band:** Narrower than Blackman due to slightly higher order. (400–500 Hz)
- **Stopband:** Moderate attenuation (~65dB)
- **Overall:** Meets all constraints.

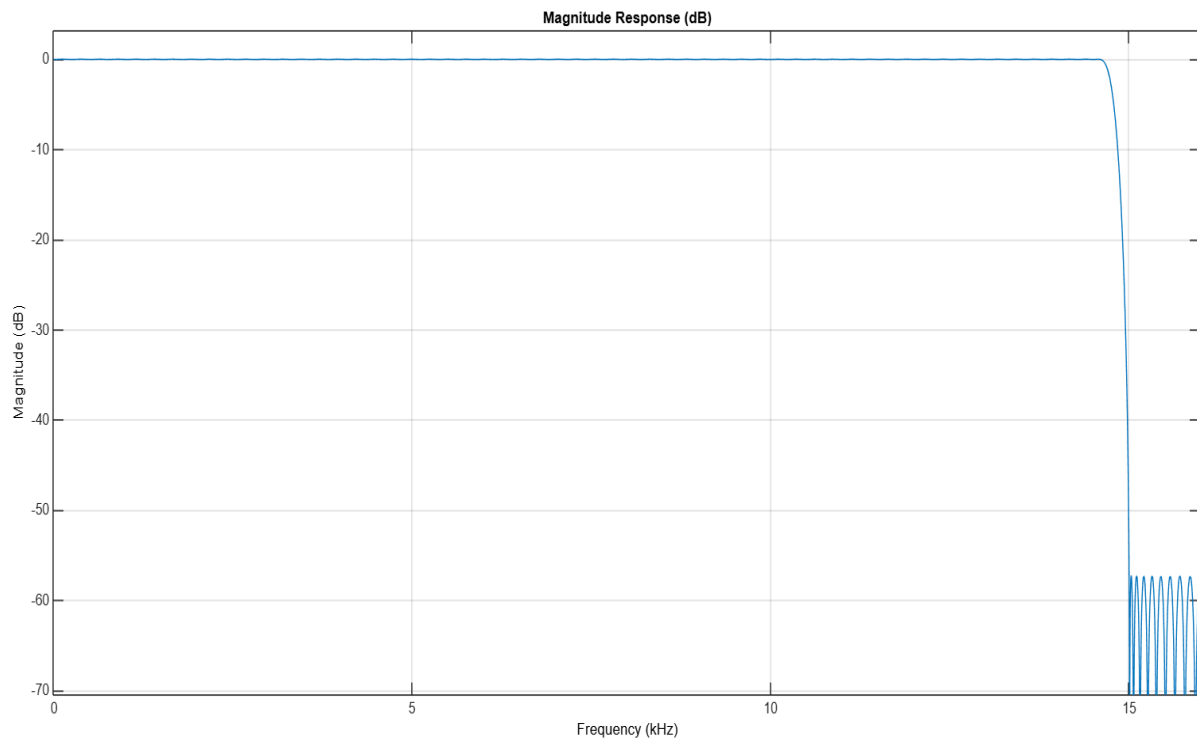
3. Least Squares filter (N=150):

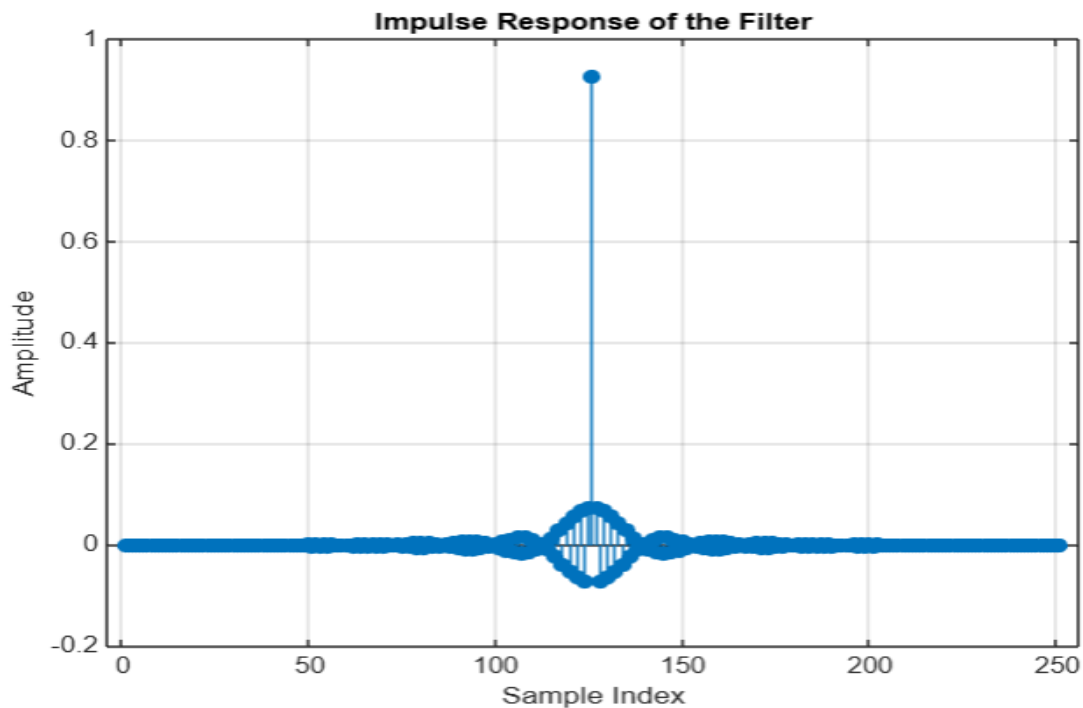


- Observations:

- **Passband: Flat on average.(<1dB)**
- **Transition Band: Sharper than both Hamming and Blackman due to optimization.(300–400 Hz)**
- **Stopband: Very good attenuation(~80–85 dB)**
- **Overall: Excellent compromise; meets all constraints effectively.**

4. Equiripple filter(N=250):





-Observation:

- **Passband:** Small, equalized ripple slightly higher than others (<1 dB).
- **Transition Band:** Sharpest among all (250–350 Hz)
- **Stopband:** Best attenuation (90–95 dB)
- **Overall:** Most effective for this application; clearly satisfies all constraints.

-Decision matrix

The decision matrix is used to compare different digital filters to choose the best one for removing the 15.2 kHz interference. Each filter is evaluated based on important criteria like how well it

removes the noise, keeps the audio clear, and avoids distortion. We assign weights to each criterion based on how important it is. Then, we rate each filter and calculate a final score. The filter with the highest score is considered the best choice for our goal.

-> Scoring Approach:

- **Passband Ripple (dB):**
 - **Ideal: 0.1 dB (best score)**
 - **Maximum acceptable: 1 dB (lower score)**
- **Stopband Attenuation (dB):**
 - **Ideal: 60 dB or higher.**
 - **The higher the attenuation , the higher the score.**
- **Transition Region Sharpness (Hz):**
 - **For Blackman and Hamming, the transition region is much wider, so their scores will be lower.**
 - **For Least Squares and Equiripple, the transition region is smaller, closer to the ideal, so they will score higher.**

->Criteria and Weights

- **Stopband Attenuation** (How well the filter attenuates the 15.2 kHz frequency): **Weight = 0.4**
- **Transition Region Sharpness** (How sharp the transition between passband and stopband is): **Weight = 0.3**

- **Passband Preservation** (How well the filter preserves frequencies in the passband): Weight = **0.2**
- **Complexity** (Number of samples): Weight=**0.1**
- The numbers 5, 4, 3, 2, and 1 in the decision matrix show how well each filter performs for each criterion:
- **5** – Excellent
- **4** – Good
- **3** – Average
- **2** – Weak
- **1** – Poor

Filter Design Decision Matrix					
Criteria	Weights	Blackman (N=180)	Hamming(N=200)	Least squares (N=150)	Equiripple (N=250)
Stopband Attenuation	0.4	3	2	4	5
Transition sharpness	0.3	2	3	4	5
Passband Preservation	0.2	5	5	4	3
Complexity(N)	0.1	4	3	5	2
Weighted score	Total	3.2	3	4.1	4.3

-Calculate the Utility Scores

Blackman Window Filter:

- **Stopband Attenuation:** $3 \times 0.4 = 1.2$

- **Transition Region Sharpness:** $2 \times 0.3 = 0.6$
- **Passband Preservation:** $5 \times 0.2 = 1$
- **Complexity:** $4 \times 0.1 = 0.4$
- **Total Utility Score:** $1.2 + 0.6 + 1 + 0.4 = \mathbf{3.2}$

Hamming Window Filter:

- **Stopband Attenuation:** $2 \times 0.4 = 0.8$
- **Transition Region Sharpness:** $3 \times 0.3 = 0.9$
- **Passband Preservation:** $5 \times 0.2 = 1.0$
- **Complexity:** $3 \times 0.1 = 0.3$
- **Total Utility Score:** $0.8 + 0.9 + 1.0 + 0.3 = \mathbf{3}$

Least Squares Filter:

- **Stopband Attenuation:** $4 \times 0.4 = 1.6$
- **Transition Region Sharpness:** $4 \times 0.3 = 1.2$
- **Passband Preservation:** $4 \times 0.2 = 0.8$
- **Complexity:** $5 \times 0.1 = 0.5$
- **Total Utility score:** $1.6 + 1.2 + 0.8 + 0.5 = \mathbf{4.1}$

Equiripple Filter:

- **Stopband Attenuation:** $5 \times 0.4 = 2.0$
- **Transition Region Sharpness:** $5 \times 0.3 = 1.5$

- **Passband Preservation:** $3 \times 0.2 = 0.6$
- **Complexity:** $2 \times 0.1 = 0.2$
- **Total Utility Score:** $2.0 + 1.5 + 0.6 + 0.2 = 4.3$

-Optimal Filter Selection for 15.2 kHz Interference Removal

The Equiripple filter is the best choice for removing the 15.2 kHz interference. It has the strongest noise reduction, the sharpest transition between passband and stopband, and keeps the passband ripple within acceptable limits. While it's slightly more complex, its performance makes it the most effective filter for this problem.

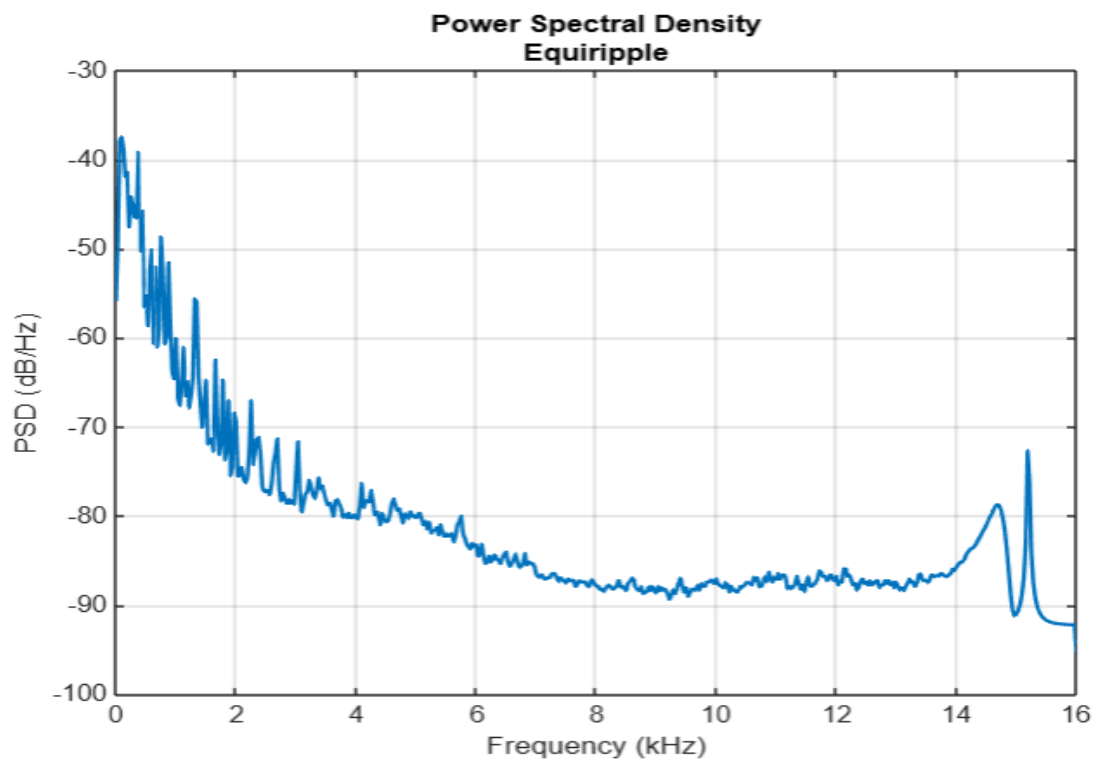
-Plotting the frequency spectrum of Equiripple filter

```
% 4.Equiripple filtering

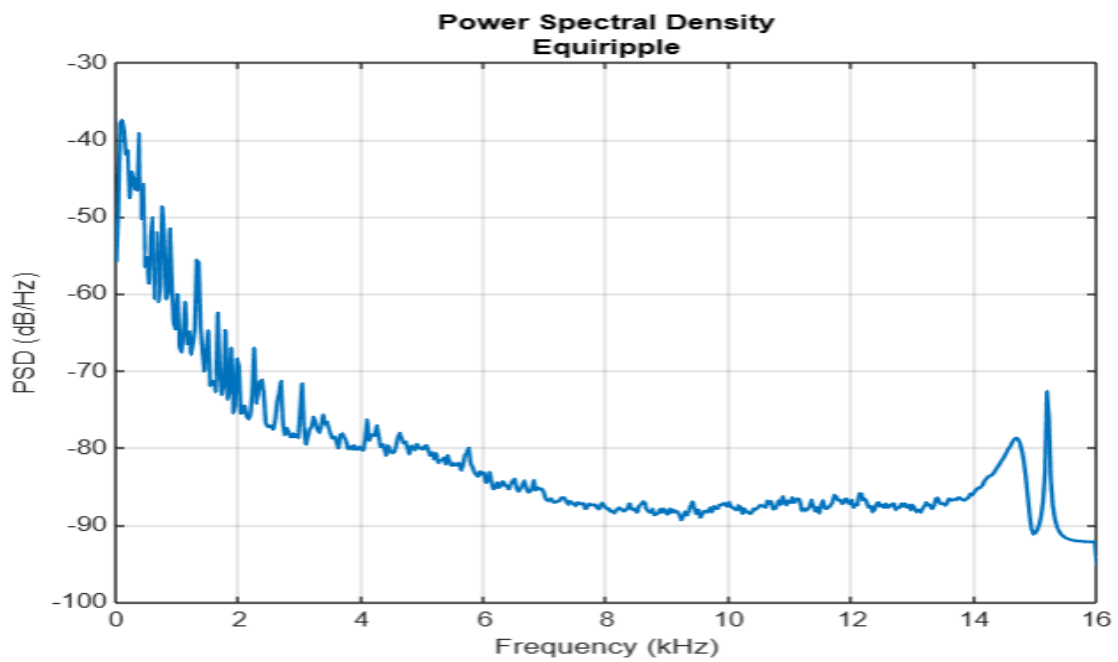
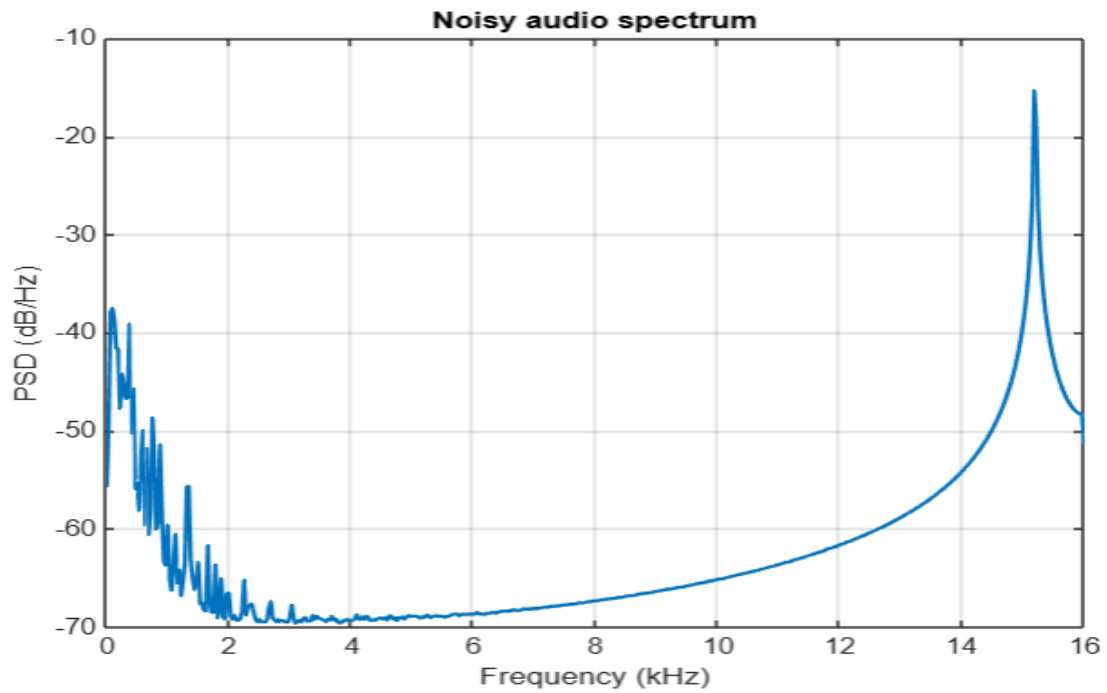
% Apply Equiripple filter using coefficients b4
Y_Filtered=filter(b4,1,Y_total);

% Plot spectrum of filtered audio
Plot_Spectrum(Y_Filtered, Fs, 1024, 1024, 'rect', 0.5);% Using 'rect' window (no specific window)

%play the filtered audio
sound(Y_Filtered,Fs)
```



-Plot spectrum of noisy signal vs filtered signal:



-Conclusion

The goal of this work was to suppress the 15.2 kHz noise in the audio signal while maintaining the quality of the original sound. Using the Equiripple filter, the noise was successfully reduced, as shown by the comparison of the noisy and filtered spectra, as well as the output audio. The filtered signal demonstrates a clear reduction in interference, with the original audio remaining mostly unchanged, effectively solving the noise suppression issue.

-Steps of solving the problem:

1)Plot the Spectrum:

- Plot the frequency spectrum of the **original audio** and the **noisy audio** to analyze the difference, particularly the noise at 15.2 kHz.

2)Choose Filters:

- Select **4 different filters** (Equiripple, Least Squares, Hamming, Blackman) as potential solutions for the noise suppression problem.

3) Compare Filters:

- Use a **decision matrix** to compare the filters based on parameters like stopband attenuation, Passband preservation, and transition sharpness.

4) Select Optimal Filter:

- Choose the filter that performs the best based on the decision matrix. This filter will be our **optimal solution**.

5) **Apply Filtering:**

- Apply the chosen filter to the noisy audio signal to **suppress the 15.2 kHz noise** while retaining the original signal's quality.

6) **Verify Results:**

- Compare the **filtered signal** with the **original audio** to ensure the noise at 15.2 kHz has been attenuated and the signal closely resembles the original.